Анализ времени компиляции C++ сервиса на userver

Постановка задачи и гипотеза

С помощью Clang Build Analyzer были проанализированы время компиляции отдельных исходных файлов, затраты на обработку заголовков и инстанцирование шаблонов при сборке сервиса, созданного с помощью генератора userver.

По результатам анализа `main.cpp` оказался наиболее затратной единицей трансляции по времени работы frontend компилятора. Дальнейший анализ трейсов с помощью Perfetto показал значительные затраты на обработку заголовка `hello_grpc.hpp`, который подключает сгенерированный gRPC-заголовок `hello_service.usrv.pb.hpp`.

Для проверки гипотезы было предложено вынести регистрацию gRPC-компонента `HelloGrpc` из `main.cpp` в `hello_grpc.cpp`. Это позволит исключить подключение `hello_grpc.hpp` из `main.cpp` и избежать повторной обработки транзитивно подключаемых сгенерированных gRPC-заголовков.

Гипотеза: предложенное изменение уменьшит время работы frontend компилятора при обработке `main.cpp` и число исходных файлов, при компиляции которых требуется обработка `hello_service.usrv.pb.hpp`.

Окружение для эксперимента

Измерения проводились на ноутбуке с процессором Intel Core i5-1145G7 и 16 ГБ оперативной памяти. ОС - Linux с ядром 6.17.0-35-generic. Сборки выполнялись через Docker Desktop в контейнере на основе Ubuntu 22.04. Для контейнера были установлены ограничения: не более четырёх CPU и 8 ГБ оперативной памяти. Использовались Clang 14, CMake, Ninja и конфигурация Release. Одновременно выполнялось не более двух задач сборки, кеширование компиляции было отключено.

Методика

В эксперименте сравнивались две версии сервиса: `origin`, в которой компонент `HelloGrpc` регистрируется в `main.cpp`, и `experiment`, в которой регистрация была вынесена в `hello_grpc.cpp`. Остальные исходные файлы и настройки сборки оставались неизменными.

Для каждой версии были выполнены три чистые сборки с удалением каталога `build-release` перед каждым запуском. Сборки чередовались в порядке `origin` → `experiment`. Трейсы компиляции собирались с помощью Clang `-ftime-trace` и анализировались через Clang Build Analyzer.

Сравнивались frontend-время компиляции `main.cpp`, суммарное frontend-время всех единиц трансляции и число обработок заголовка `hello_service.usrv.pb.hpp`. Для сравнения временных показателей использовались медианы трёх измерений каждой версии.

Эксперимент воспроизводится командой `make reproduce`. Результаты анализа сохраняются в каталоге `results/`.

Результаты

Показатель	origin	experiment	Изменение
Frontend `main.cpp`	4684 мс	3343 мс	–28,6%
Суммарное время frontend	24,2 с	23,6 с	–2,5%
Обработка `hello_service.usrv.pb.hpp`	4153 мс	3156 мс	–24,0%
Число обработок заголовка	3	2	–33,3%

В таблице приведены медианы трёх измерений. Суммарное время frontend — сумма времени обработки всех компилируемых файлов, а не реальная продолжительность сборки.

Гипотеза подтвердилась: число обработок hello_service.usrv.pb.hpp сократилось с трёх до двух, а медианное frontend-время main.cpp уменьшилось на 28,6%. При этом суммарное frontend-время снизилось лишь на 2,5%, поскольку оптимизация затронула только одну единицу трансляции.

Ограничения и что не получилось

Не удалось продемонстрировать столь же заметное улучшение на уровне всего проекта: медианное суммарное frontend-время сократилось лишь на 2,5%, что сопоставимо с разбросом измерений.

Реальное время полной сборки не измерялось, поэтому оценить влияние оптимизации на её продолжительность нельзя. Кроме того, параллельная компиляция могла влиять на результаты из-за конкуренции за ресурсы.

Выводы

Перенос регистрации gRPC-компонента HelloGrpc из `main.cpp` в `hello_grpc.cpp` позволил исключить повторную обработку тяжёлых сгенерированных gRPC-заголовков в `main.cpp`.

Гипотеза подтвердилась: медианное frontend-время `main.cpp` сократилось на 28,6%, а число обработок `hello_service.usrv.pb.hpp` уменьшилось с трёх до двух. При этом эффект на уровне всего проекта оказался небольшим: медианное суммарное frontend-время сократилось на 2,5%.